



Tecnológico de Monterrey

Campus Monterrey

Act 2.3 - Evidence 1

Ethiel Favila Alvarado
A00844789

Luis Salomón Flores Ugalde
Programming of Data Structures and Fundamental Algorithms
Group 610

24 of September 2026

Investigation:

Importance and efficiency of the sorting and searching algorithms

When working with large unorganized data structures being able to organize and search in the data is crucial. In this case, we worked with security logs, but the same is true for any time of data.

First of all we need to sort the data, in ascending or descending order. This is important since it facilitates the latest search and makes it possible to search in ranges; furthermore, it allows us to identify patterns and trends in data (GeeksforGeeks, 2026). When working with large data files, finding the most time efficient algorithm is crucial. While low-efficiency one might me fine with few entries, in one like the one we used (around sixteen thousand entries) economizing time is relevant. As such, using algorithms perform operations more efficiently. In this case, Merge Sort was utilized which is an efficient method, it has a time complexity of $O(n \log n)$ since it isolates all the elements regardless of if they are organized or not, and a plus that means that the best case, worst case and average case all have the same complexity (GeeksforGeeks, 2025). This means, we can plan the time complexity with certainty each time.

When data is organized, it is important to be able to look for certain queries or ranges. Doing so line by line is very inefficient, thus we use search algorithms. In this case we used binary search, which can find positions in a sorted vector, it does so by dividing the research interval in half until the target is found (Revanda, N. F, 2025). It is important to note that it only works in sorted vectors, but it's faster since it uses the divide-and-conquer approach (Revanda, N. F, 2025).

When combining the best sorting and search algorithms we can economize the time and memory cost.

Technical report:

Problem and relevance:

What is the problem and why is it relevant?

The problem is that we need to identify, organize and analyze failed authentication attempts and unauthorized access requests logged in the system. It is important we analyze them properly as it is a security matter. The unsorted security logs contained a lot of data which, without processing, is difficult to detect breaches

or malicious IP addresses. To organize them better increased the chances of bettering security.

How was the problem solved?

To solve this problem we created the Keeper class which helps organize and filter the data. IT has various functions that handle the unsorted data. First, read() classifies each line separating the parts like month day time and ip, then stores the rest of the text in lines. Sorting rounds merge sort on the dates, moving everything so it's in the correct order. searchRecords() allows us to use the now classified data and look for logs between specific dates. It uses binary search to find them, so it only works once the data has been organized. store() writes the results in a text for easy access.

Data structures and algorithms:

What data structures and algorithms were used?

We used several vectors to have the lines (text), the dates and the results. Moreover, we used a linear lookup in the array in monthtoNumber. Merge sort and binary search were also used. Read and store where also used.

How were they implemented and why were they selected?

Vectors were used instead of arrays due to familiarity. However we used two different ones in order to keep a better order and corresponding dates and text without having to use structs. Both vector dates and lines match in the index. MErge sort was used as is one of the best sorting algorithms with a low time complexity. Worst case we got $O(n \log n)$ but best case was $O(n^2)$. On the other hand, binary search was used because it was fast and the data was already searched.

What are their advantages and disadvantages?

Using the integer dates as “keys” makes comparisons easy. Merge sort is also easy to use, plus it enhances binary search. However, we recognize some disadvantages. First of all, merge sort uses a lot of memory because it creates temporary vectors. Plus, there's always the possibility that the temporary vectors get misaligned and then the data is all wrong.

Computational complexity:

What is their computational complexity in the best and worst cases, and what determines it?

Algorithm	Best	Worst	What determines it?
monthToNumber	$O(1)$	$O(1)$	Best case scenario January is found in the first iteration, worst case it's December or invalid. But still, the size is determined to twelve (for each month) so it stays constant.
read	$O(n)$	$O(n)$	Reads the number of n lines, each line has a constant work to format.
binarySearch	$O(\log n)$	$O(\log n)$	The loop runs until the lower end equals the high end. Normally, $O(1)$ is the best case; however, on our code specifically, it does not because we don't stop when we find the date. This is because many records might have the same date, we don't need a match, we need the first one. Thus, we keep going.
mergeSort and merge	$O(n \log n)$	$O(n \log n)$	The recursion splits the vectors until they are alone, no matter how many there are or if they are messy or already in order. Merge then always copies every element of the range.
searchRecords	$O(1)$	$O(\log n + n)$ Or $O(k)$	There are two binary records each with a cost of $O(\log n)$ so that's the worst case and normal case. Or if every record is in the range we get $O(k)$ where k is the number of records (because we go through everyone). $O(1)$ is the best case in case the range is invalid and we don't use binary search.
store	$O(1)$	$O(n)$	If the number of records is 0 then we get $O(1)$ if not $O(k)$ with k being the size of results
size	$O(1)$	$O(1)$	Calls the function size()

Improvements

What possible improvements could be implemented?

Improvements mostly include the handling of memory, merge sort created new temporary vectors in each merge, that meant a lot of space and copied strings. We didn't use pointers, so we probably should improve the use of memory. Also, we used a parallel vector, but it could've been easier with a struct. Thai wasn't quite done as we weren't sure how to use sorting algorithms there, but implementing them would be a great improvement.

As a side note, validations of data both read in with inputs could be an improvement that would help make the code neater.

References

GeeksforGeeks. (2026, January 20). Applications, advantages and disadvantages of sorting algorithms.
<https://www.geeksforgeeks.org/dsa/applications-advantages-and-disadvantages-of-sorting-algorithm/>

GeeksforGeeks. (n.d.). Time and space complexity analysis of merge sort.
<https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-merge-sort/>

Revanda, N. F. (2025, July 23). Linear vs binary search: Which one is more efficient? Medium.
<https://medium.com/@nrevanda/linear-vs-binary-search-which-one-is-more-efficient-9c46f0a52967>